

DPEL Report on NETFLIX SERIES INVESTIGATION

By-Naman Gupta 25070126111

&

Taishka Gite 25070126248

1. Introduction

This report is about the analysis of a dataset and its understanding, preprocessing and exploratory data analysis (EDA). It looks into the structure and cleaning the dataset. The aim is to transform the information into a structured format so that we can use this information for decision making and further be machine learning used.

A) Data Understanding

What we first do is to understand the dataset. We try to understand the dataset as a whole. In this step, we look at the number of rows and columns in the dataset. Column names.

Types of data (numerical, categorical, datetime)

This will give us an understanding of how the data needs to be processed.

2. Problem Statement (Real-World Relevance)

The purpose of this analysis is to investigate the dataset and to uncover patterns, trends, and relationships among the variables. In practice, such analysis can aid organizations in making better decisions, enhancing performance, and forecasting the future. For instance, the analysis of content datasets can unveil user interests and enhance recommendation systems.

3. Dataset Description

Attributes / Features-

The dataset contains several features which indicate different aspects of the data:

- a) Categorical features (type, category, country)
- b) Numerical features (duration, year)

Each feature indicates certain characteristics of the dataset.

Dataset Size-

The dataset has:

- a) **Rows:** the number of data entries
- b) **Columns:** the number of features for each entry

The dataset size indicates the level of processing and analysis needed.

Target variable-

If the dataset is used for predictions, then the target variable is the one that we want to predict.

If it is not possible to define a target variable, then we can only perform an exploratory analysis.

4. Initial Observations-

Data quality issues found-

While exploring the data, you might find the following data quality issues:

- a) Missing values in some columns
- b) Duplicate records
- c) Inconsistent formatting (e.g. mixed text formats)
- d) Missing values / duplicates / incorrect entries-
 - 1) Missing values: Empty or null values in the dataset
 - 2) Duplicates: Redundant rows that can influence analysis
 - 3) Incorrect entries: Invalid or inconsistent data (e.g. wrong formats, spelling mistakes)

These will have to be dealt with before analysis.

B) Data Preprocessing-

This step is to clean and transform the data so it is suitable for analysis.

5. Handling Missing Values-

Mean / Median / Mode / Deletion-

Different methods used depending on data type:

1. **Mean:** Numerical data, replacing missing values with the average
2. **Median:** Skewed data, to prevent skew
3. **Mode:** Categorical data (most frequently used value)

Deletion: If the proportion of missing values is small, rows are removed.

Justification of method selected-

Mean/median allows for maintaining number of rows and distribution.

Mode is correct for categorical data.

Deletion only used when proportion of missing data is small.

6. Data Cleaning-

Removing duplicate rows-

Duplicate rows are eliminated to prevent redundancy and bias in analysis.

Correcting incorrect values-

Incorrect or inconsistent values are rectified by:

1. Standardizing text (e.g., consistent case)
2. Correcting formatting issues
3. Correcting incorrect data types

Columns are converted to the correct data type:

1. Dates → datetime
2. Numerical values → int/float
3. Categories → object/category

This allows for accurate analysis.

Data Cleaning

```
# BEFORE cleaning
print("Duplicates BEFORE:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

# AFTER cleaning
print("Duplicates AFTER:", df.duplicated().sum())

# Fix column names
df.columns = df.columns.str.strip()

print("\nUpdated Column Names:")
print(df.columns)

# Show updated dataset
print("\nFirst 5 rows after cleaning:")
print(df.head())
```

Duplicates BEFORE: 0

Duplicates AFTER: 0

Updated Column Names:

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
      'release_year', 'duration', 'description', 'genre'],
      dtype='object')
```

First 5 rows after cleaning:

	show_id	type	title	director
0	s1	TV Show	3%	NaN
1	s2	Movie	7:19	Jorge Michel Grau
2	s3	Movie	23:59	Gilbert Chan
3	s4	Movie	9	Shane Acker
4	s5	Movie	21	Robert Luketic

	cast	country
0	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil

7. Feature Transformation-

Feature transformation is the process of converting or creating new features for analysis.

Examples:

1. extracting year from date columns
2. creating new categories from existing data
3. transforming text into useful numeric values

This step transforms the dataset into more useful data.

Feature Transformation

```
# Check columns first
print("Columns in dataset:\n", df.columns)

if 'date_added' in df.columns:
    df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
    df['year_added'] = df['date_added'].dt.year

    print("\nFeature Transformation Applied: 'year_added' created")

else:
    print("\nColumn 'date_added' not found → skipping transformation")

# Show result
print("\nFirst 5 rows after transformation:")
print(df.head())
```

```
Columns in dataset:
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
      'release_year', 'duration', 'description', 'genre'],
      dtype='object')
```

```
Feature Transformation Applied: 'year_added' created
```

```
First 5 rows after transformation:
```

	show_id	type	title	director	\
0	s1	TV Show	3%		NaN
1	s2	Movie	7:19	Jorge Michel Grau	
2	s3	Movie	23:59	Gilbert Chan	
3	s4	Movie	9	Shane Acker	
4	s5	Movie	21	Robert Luketic	

	cast	country	\
0	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	
1	Demián Bichir, Héctor Bonilla, Oscar Serrano, ...	Mexico	
2	Tedd Chan, Stella Chung, Henley Hii, Lawrence ...	Singapore	
3	Elijah Wood, John C. Reilly, Jennifer Connelly...	United States	
4	Jim Sturgess, Kevin Spacey, Kate Bosworth, Aar...	United States	

8. Encoding-

Label Encoding-

Transform categorical values into numeric labels.

Appropriate for ordinal data where order is relevant.

Encoding

1. Label Encoding

```
from sklearn.preprocessing import LabelEncoder

df_label = df.copy()

# Select categorical columns
cat_cols = df_label.select_dtypes(include=['object']).columns

print("Categorical Columns:\n", cat_cols)

# Apply Label Encoding
le = LabelEncoder()

for col in cat_cols:
    df_label[col] = le.fit_transform(df_label[col])

print("\nAfter Label Encoding:\n")
print(df_label.head())
```

```
Categorical Columns:
Index(['show_id', 'type', 'title', 'director', 'cast', 'country',
      'description', 'genre'],
      dtype='object')
```

After Label Encoding:

	show_id	type	title	director	cast	country	date_added	release_year	\
0	0	1	82	4049	3160	6	2020-08-14	2020	
1	1111	0	124	1840	1589	42	2016-12-23	2016	
2	2222	0	67	1289	6200	60	2018-12-20	2011	
3	3333	0	126	3445	1788	75	2017-11-16	2009	
4	4444	0	60	3176	2924	75	2020-01-01	2008	

One-Hot Encoding (when applicable)-

Transform categoricals into individual binary columns.

Appropriate for nominal data where no order is relevant.

Encoding allows for categorical values to be used in analysis and models.

2. One-Hot Encoding

```
# Copy dataset
df_ohe = df.copy()

# Select categorical columns
cat_cols = df_ohe.select_dtypes(include=['object']).columns

print("Categorical Columns:\n", cat_cols)

# Apply One-Hot Encoding
df_ohe = pd.get_dummies(df_ohe, columns=cat_cols).astype(int)

print("\nAfter One-Hot Encoding:\n")
print(df_ohe.head())

print("\nNew Columns Created:\n")
print(df_ohe.columns)
```

```
Categorical Columns:
Index(['show_id', 'type', 'title', 'director', 'cast', 'country',
      'description', 'genre'],
      dtype='object')
```

After One-Hot Encoding:

	date_added	release_year	duration	year_added	show_id_s1	\
0	1597363200000000000	2020	4	2020	1	
1	1482451200000000000	2016	93	2016	0	
2	1545264000000000000	2011	78	2018	0	
3	1510790400000000000	2009	80	2017	0	
4	1577836800000000000	2008	123	2020	0	

	show_id_s10	show_id_s100	show_id_s1000	show_id_s1001	show_id_s1002	\
0	0	0	0	0	0	
1	0	0	0	0	0	
2	0	0	0	0	0	

9. Scaling / Normalization

Standardization / Normalization

1. Standardization: data scaling to mean = 0, stdev = 1
2. Normalization: scaling values between 0 and 1

✓ Scaling / Normalization

1. Standardization(Z-score Scaling)

```
[ ] from sklearn.preprocessing import StandardScaler

# Copy dataset
df_std = df.copy()

# Select numerical columns
num_cols = df_std.select_dtypes(include=['int64', 'float64']).columns

print("Numerical Columns:\n", num_cols)

# Apply Standardization
scaler = StandardScaler()
df_std[num_cols] = scaler.fit_transform(df_std[num_cols])

# Show output

print("\nAfter Standardization:\n")
print(df_std.head())
```

✓ Numerical Columns:
Index(['release_year', 'duration', 'year_added'], dtype='object')

After Standardization:

	show_id	type	title	director	\
0	s1	TV Show	3%		NaN
1	s2	Movie	7:19	Jorge Michel Grau	
2	s3	Movie	23:59	Gilbert Chan	
3	s4	Movie	9	Shane Acker	
4	s5	Movie	21	Robert Luketic	

3.

2. Normalization(Min-Max Scaling)

```
from sklearn.preprocessing import MinMaxScaler

# Copy dataset
df_norm = df.copy()

# Select numerical columns
num_cols = df_norm.select_dtypes(include=['int64', 'float64']).columns

print("Numerical Columns:\n", num_cols)

# Apply Normalization
scaler =

df_norm[num_cols] = scaler.fit_transform(df_norm[num_cols])

# Show output
print("\nAfter Normalization:\n")
print(df_norm.head())
```

```
Numerical Columns:
Index(['release_year', 'duration', 'year_added'], dtype='object')
```

```
After Normalization:
```

	show_id	type	title	director	\
0	s1	TV Show	3%		NaN
1	s2	Movie	7:19	Jorge Michel Grau	
2	s3	Movie	23:59	Gilbert Chan	
3	s4	Movie	9	Shane Acker	
4	s5	Movie	21	Robert Luketic	

	cast	country	\
0	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	
1	Demián Bichir, Héctor Bonilla, Oscar Serrano, ...	Mexico	
2	Tedd Chan, Stella Chung, Henley Hii, Lawrence ...	Singapore	

Reasoning behind the method used-

Standardization is used for normal distributed data.

Normalization is used for data with different ranges.

A step to allow a fair comparison between features.

C) EDA and Visualizations-

10.Univariate Analysis

Univariate analysis is an analysis that analyzes each variable individually.

It is used for:

1. Determining distribution
 2. Identifying outliers
 3. Determining central tendency (mean, median, mode)
-

11.Bivariate Analysis

Bivariate analysis is an analysis of two variables.

It is used for:

1. Finding correlations
 2. Recognizing trends
 3. Analyzing variables
-

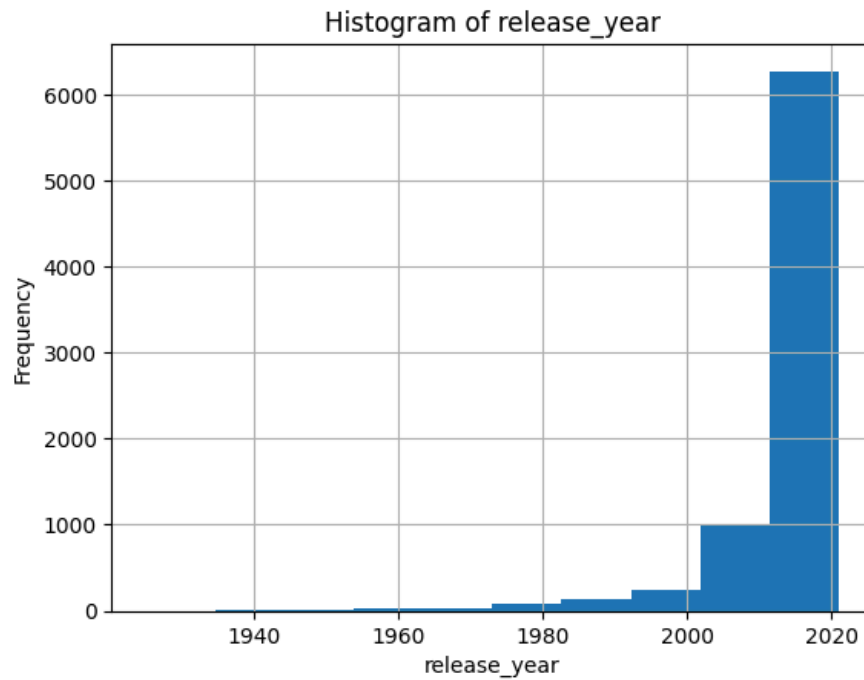
12.Visualizations with Insights

• Histogram

The histogram displays the frequency distribution of numerical data.

Insight: Understand if the data is normal, skewed or uniform.

*** Selected Column for Histogram: release_year

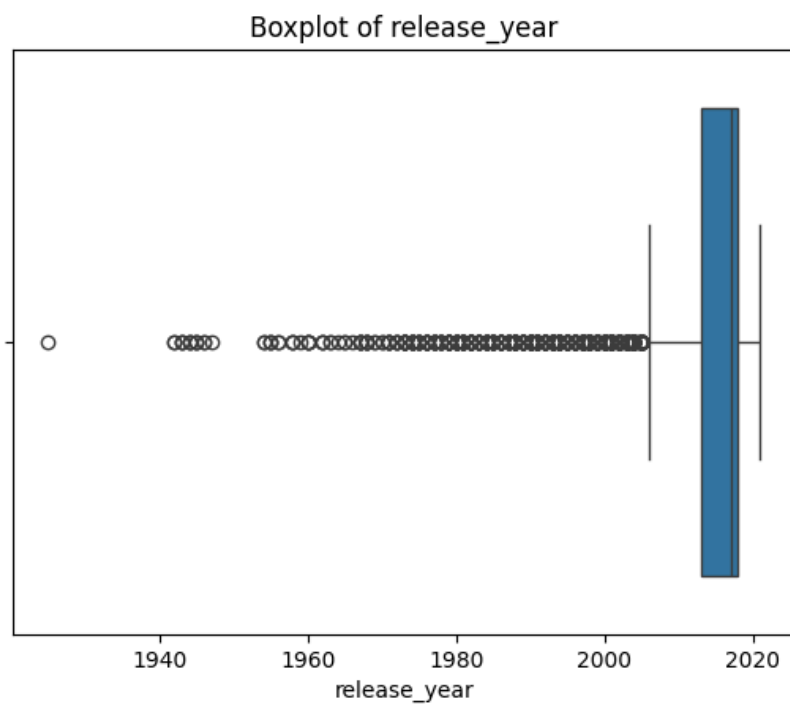


• Boxplot

The boxplot displays the quartiles and outliers.

Insight: Identify extreme values and spread of data.

*** Selected Column for Boxplot: release_year

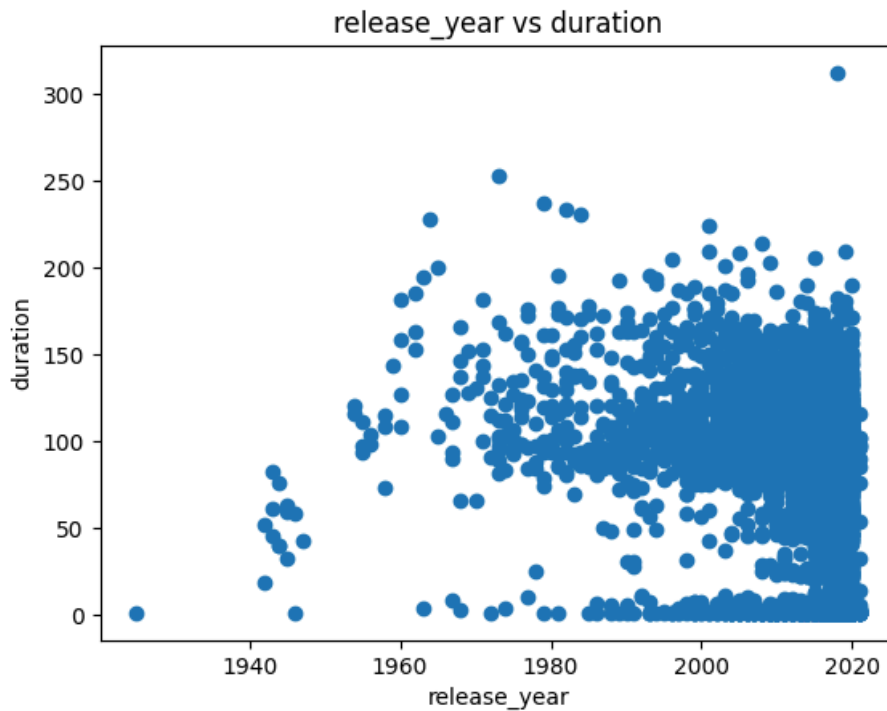


• Scatter Plot

The scatter plot displays the relationship between two numerical variables.

Insight: Identify a positive, negative or no correlation.

*** Scatter Plot between release_year and duration



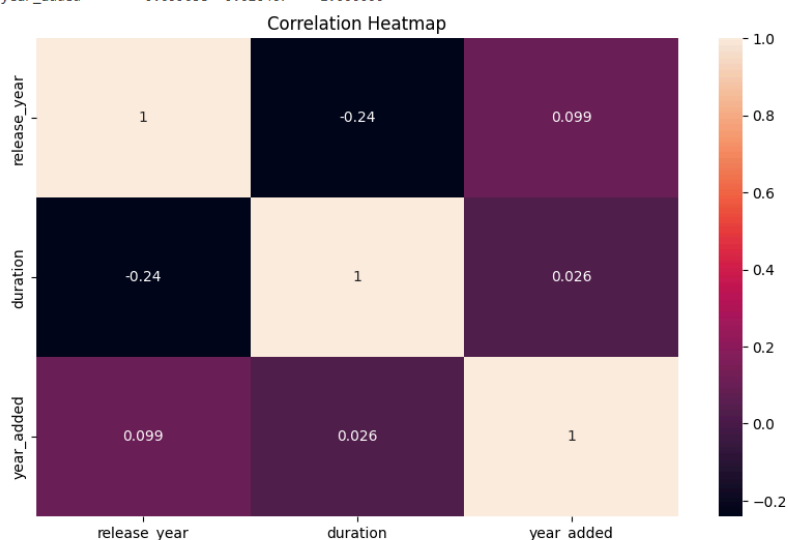
• Heatmap

The heatmap displays the correlation between features using colors.

Insight: Identify the strong and weak relationships between features.

*** Correlation Matrix:

	release_year	duration	year_added
release_year	1.000000	-0.240836	0.099058
duration	-0.240836	1.000000	0.026467
year_added	0.099058	0.026467	1.000000



Final Conclusion

A comprehensive analysis of a real-world dataset: Understanding its challenges and transformations. The dataset was meticulously examined by gaining insights, preprocessing, and visualization. Missing and duplicate values were addressed and transformed using various techniques. Encoded and scaled the data features to enhance their representation. Generated visualizations that provided valuable information on patterns and relationships within the data.
